ANURAG TRIVEDI

DEVOPS ENGINEER

+91-6388448992 anuragtrivedi1523@gmail.com Ghaziabad, UP

Certifications

- AWS Certified DevOps Engineer – Professional
- Microsoft Certified: DevOps Engineer Expert

Skills

- **Cloud Platforms:** AWS (EC2, S3, VPC, IAM, ALB, Route53, CloudFront, RDS, ElastiCache, CloudWatch, CodePipeline, API Gateway, Kafka, Lex), Azure, GCP
- **CI/CD & Automation:** Jenkins, GitHub Actions, GitLab CI/CD, ArgoCD, Bamboo, Spinnaker
- **Infrastructure as Code (IaC):** Terraform, Ansible, AWS CloudFormation, Chef, Puppet
- **Containerization & Orchestration:** Docker, Podman, Kubernetes (EKS, AKS, GKE), Helm, Istio, Linkerd
- **Monitoring & Observability:** Prometheus, Grafana, ELK/EFK Stack, Datadog, AWS CloudWatch, Nagios
- **Version Control & SCM:** Git, GitHub, GitLab, Bitbucket
- **Scripting & Programming:** Python, Bash, PowerShell, Groovy, YAML, JSON
- **Security & DevSecOps:** SAST, DAST, SCA, IaC Security Scanning, Vault, Secrets Management, RBAC, IAM Policies
- **Networking & Hybrid Cloud:** VPC Peering, VPN, Transit Gateway, PrivateLink, DNS, Load Balancers
- **Operating Systems:** Linux (RHEL, Ubuntu, CentOS), Windows Server
- **Databases & Caching:** MySQL, PostgreSQL, MongoDB, Redis, ElastiCache
- **Project Management & Collaboration:** Agile/Scrum, Jira, Confluence, Slack, MS Teams

Summary

- **DevOps & Cloud Engineer with 3.5+ years of experience** specializing in **Automation**, **Infrastructure as Code (IaC)**, **CI/CD Engineering, and Cloud-Native Deployments**. Strong hands-on expertise in **AWS and Azure**, designing **scalable, secure, and highly available** cloud architectures.
- Proven experience architecting and optimizing **CI/CD pipelines using Jenkins, GitHub Actions, GitLab CI/CD, and ArgoCD,** significantly reducing release cycles and deployment failures.
- Extensive experience implementing **Infrastructure as Code with Terraform, Ansible, and AWS CloudFormation**, enabling repeatable, version-controlled infrastructure provisioning across environments.
- Strong background in **Containerization & Orchestration** using **Docker, Podman, and Kubernetes (EKS, AKS, GKE)**, supporting **microservices architecture, rolling updates, blue-green deployments, and zero-downtime releases**.
- Hands-on expertise with core **AWS services (EC2, S3, VPC, IAM, RDS, ElastiCache, API Gateway, CloudFront, Kafka, Lex)** and implementing secure **Hybrid/Multi-Cloud Networking (VPC Peering, VPN, Transit Gateway, PrivateLink)**.
- Experienced in integrating **DevSecOps practices** including **SAST, DAST, SCA, and IaC Security Scanning**, ensuring secure and compliant delivery pipelines.
- Strong in **Monitoring & Observability** using **Prometheus, Grafana, ELK/EFK Stack, Datadog, and AWS CloudWatch**, reducing MTTR and improving SLA compliance.
- Automated **Disaster Recovery, Backup Strategies, Auto-Scaling, and Cost Optimization**, improving business continuity and reducing cloud spend.
- Proficient in **Python, Bash, PowerShell, and Groovy scripting** for automation and pipeline development.
- Certified in **AWS and HashiCorp**, with a strong focus on **scalability, reliability, cost efficiency, and DevOps culture enablement.**

Education

Bachelor's in Computer Applications - Christ (Deemed to be University) - (2020 - 2023)

Experience

DevOps Engineer

June 2022 - Present

Orbex Pvt. Ltd.

- Designed and optimized **CI/CD pipelines** using **Jenkins, GitHub Actions, and GitLab CI/CD**, reducing deployment time by **70%** and improving release stability.
- Automated **Infrastructure as Code (Terraform, Ansible, CloudFormation)** across SIT, UAT, and Production, eliminating configuration drift and manual provisioning errors.
- Implemented **Docker containerization** and deployed microservices on **Kubernetes (EKS, AKS)** with rolling updates and zero-downtime releases.
- Architected scalable **AWS infrastructure** (EC2, VPC, IAM, S3, RDS, CloudFront, Route53, ElastiCache, API Gateway), ensuring high availability and performance.
- Embedded **DevSecOps practices** (SAST, DAST, SCA, IaC scanning) into pipelines to enforce compliance and reduce security risks.
- Built centralized **monitoring & observability using Prometheus, Grafana, ELK/EFK, CloudWatch**, reducing MTTR by **40%.**
- Implemented **cost optimization strategies** (auto-scaling, right-sizing, cleanup), reducing cloud spend by **~20%**.
- Collaborated with Dev & QA teams to streamline deployments, perform RCA, and maintain **24/7 uptime and SLA compliance**.

Projects

Project 1: Enterprise API Platform Modernization with Kubernetes & Service Mesh

Role: DevOps Engineer

Tech Stack:

- AWS (EKS, ALB, IAM, VPC, RDS, S3, CloudWatch), Terraform, Ansible, Jenkins, GitHub Actions, Docker, Helm, Istio, Prometheus, Grafana, ELK

Description:

Led the modernization of an enterprise **API management platform** handling high-volume B2B and mobile traffic. Migrated legacy API services from VM-based deployments to a **containerized microservices architecture on Amazon EKS**, implemented **Infrastructure as Code with Terraform**, and introduced **service mesh (Istio)** for secure service-to-service communication, traffic control, and observability. Designed secure CI/CD pipelines and centralized monitoring to ensure performance, scalability, and compliance.

Roles & Responsibilities:

- Migrated legacy API services from EC2/IIS-based deployments to **Dockerized microservices on AWS EKS**.
- Designed modular **Terraform infrastructure** for VPC, EKS, RDS, ALB, IAM roles, and networking components.
- Automated OS hardening, patching, and runtime configuration using **Ansible playbooks**.
- Implemented **Helm-based deployments** with rolling updates and version-controlled releases.
- Integrated **Istio service mesh** for traffic routing, mutual TLS (mTLS), and fault injection testing.
- Designed **secure CI/CD pipelines** using Jenkins + GitHub Actions with build, test, scan, and deploy stages.
- Embedded **DevSecOps controls** (SAST, container scanning, IaC scanning) into pipelines.

Impact / Achievements:

- Improved API availability from **98.5% to 99.95%** post-migration.
- Reduced deployment time by **60%** through automated pipelines and Helm-based releases.
- Increased platform scalability to handle **2x peak traffic** without downtime.
- Reduced infrastructure cost by **20–25%** via containerization and autoscaling.
- Improved MTTR by **45%** through centralized monitoring and alerting.

Projects

Project 2: Real-Time Data Processing & Event Streaming Platform (Kafka + Kubernetes)

Role: DevOps Engineer

Tech Stack:

- AWS (EKS, MSK/Kafka, RDS, S3, IAM, CloudWatch, Route53), Terraform, Ansible, Jenkins, GitHub Actions, Docker, Helm, Prometheus, Grafana, ELK, Trivy, SonarQube

Description:

Designed and implemented a scalable **real-time data processing platform** to handle high-volume transactional and event-driven workloads. The system leveraged **Apache Kafka (AWS MSK)** for event streaming and deployed containerized microservices on **Amazon EKS**. Built secure, automated CI/CD pipelines and infrastructure using **Terraform and Ansible**, enabling continuous delivery of streaming services with strong observability and fault tolerance.

Roles & Responsibilities:

- Architected CI/CD pipelines using **Jenkins and GitHub Actions** for build, test, security scan, and deployment of streaming microservices.
- Provisioned infrastructure using **Terraform** for EKS clusters, MSK (Kafka), RDS, IAM roles, and networking components.
- Containerized event-processing services using **Docker** and deployed **via Helm charts** to EKS.
- Configured and managed **Kafka topics, partitions, replication factors**, and consumer scaling strategies.
- Integrated **Trivy and SonarQube** for container and code security scanning within pipelines.
- Automated configuration and runtime secrets management using **Ansible and AWS Secrets Manager**.
- Implemented **blue-green and rolling deployment strategies** to avoid service disruption.

Impact / Achievements:

- Enabled processing of **millions of real-time events per day** with high reliability.
- Reduced deployment time by **65%** through fully automated CI/CD pipelines.
- Improved system scalability, supporting **2x traffic growth** without infrastructure redesign.
- Reduced infrastructure cost by **30%** using autoscaling and optimized Kafka partitioning.
- Improved MTTR by **45%** through centralized monitoring and lag-based alerting.
- Strengthened security posture by integrating automated container and dependency scanning.
- Reduced manual provisioning effort from days to **under 1 hour** using Terraform.
- Achieved **99.9% platform uptime** with automated failover and rolling upgrades.

Project 3: Real-Time Monitoring & Logging for SaaS Applications

Role: DevOps Engineer

Tech Stack:

- **Cloud & Infrastructure:** AWS (EC2, S3, VPC, IAM, CloudWatch)
- **Monitoring & Logging:** Prometheus, Grafana, ELK Stack (Elasticsearch, Logstash, Kibana), CloudWatch
- **CI/CD & Automation:** Jenkins, Terraform, Ansible
- **Containers & Orchestration:** Docker, Kubernetes

Description:

Implemented a **centralized monitoring and logging** solution for a distributed SaaS application, enabling real-time observability, proactive incident detection, and rapid troubleshooting. Deployed Prometheus and Grafana for metrics monitoring, and set up the ELK stack for centralized log management. Integrated automated alerts and dashboards into CI/CD workflows for both infrastructure and application layers.

Projects

Roles & Responsibilities:

- Designed and deployed **centralized monitoring and logging architecture** using Prometheus, Grafana, and ELK Stack.
- Integrated monitoring pipelines with **Jenkins** and automated deployment of dashboards and alerts.
- Configured containerized microservices using **Docker** and managed deployments with Kubernetes.
- Automated provisioning of monitoring infrastructure with **Terraform** and configuration management via Ansible.
- Implemented alerting and notification workflows to notify teams of critical incidents in real-time.
- Collaborated with development and QA teams to instrument application metrics for enhanced observability.

Impact / Achievements:

- Reduced **Mean Time to Resolution (MTTR) by 40%** through proactive monitoring and alerting.
- Improved application reliability and availability by implementing **real-time dashboards and alerts.**
- Standardized log aggregation and monitoring for multiple microservices, improving debugging efficiency.
- Enabled rapid response to infrastructure or application issues, minimizing downtime for end-users.

Project 4: Infrastructure Modernization with GitOps & GitHub Actions

Role: DevOps Engineer

Tech Stack:

- AWS (EKS, ALB, RDS, S3, CloudTrail), Terraform, Ansible, GitHub Actions, Helm, Docker, ArgoCD (GitOps), SonarQube, Trivy, Prometheus, Grafana

Description:

The company was running a **legacy monolithic app** on VMs with manual deployments, causing **slow releases and frequent outages**. I led the modernization initiative to **migrate to AWS EKS**, implement **GitOps workflows**, and integrate **GitHub Actions** for automation. We used **Terraform** for AWS infrastructure (network, RDS, ALB, EKS), **Ansible** for configuration, and **GitHub Actions** pipelines for CI/CD. Deployment was handled with **Helm charts. Prometheus & Grafana** were added for monitoring, and **cross-region replication** ensured disaster recovery.

Roles & Responsibilities:

- Migrated a **legacy monolithic app to microservices** deployed on Kubernetes.
- Built **Terraform IaC** for AWS infra (networking, IAM, EKS, RDS, ALB).
- Implemented **GitHub Actions workflows** for build, test, security scan, and deployment.
- Integrated **SonarQube and Trivy scans** to enforce quality gates in pipelines.
- Designed **Helm charts** for consistent deployment of microservices.
- Adopted **GitOps with ArgoCD** to ensure version-controlled deployments.
- Automated **OS patching, secrets management, and node scaling** with Ansible.
- Configured **multi-region replication** for HA and DR readiness.
- Conducted **chaos testing** for failover validation.

Impact / Achievements:

- Reduced infra provisioning time from **weeks → <1 hour**.
- Improved uptime from **96% → 99.9%** by migrating to Kubernetes.
- Increased deployment success rate by **70%** with automated rollbacks.
- Reduced **manual operations by 80%,** freeing engineers for innovation.
- Improved **code quality and security** by blocking insecure builds.
- Reduced infra costs by **30%** with autoscaling and optimized cluster sizing.
- Achieved **faster DR recovery (RTO reduced by 50%)**.

Projects

Project 5: Centralized Monitoring & Auto-Healing for Cloud Workloads

Role: DevOps Engineer
Tech Stack:

- AWS,, Kubernetes, Prometheus, Grafana, Ansible, ELK Stack, Jenkins, Slack, Chaos Toolkit

Description:

Our applications spanned **multiple Kubernetes clusters** across AWS and Azure, but monitoring was fragmented and outages were difficult to track. I built a **centralized observability and auto-healing system** using **Prometheus, Grafana**, **Ansible, and Jenkins**. We used **Prometheus federation** for multi-cluster metrics, **Grafana dashboards** for visualization, and **auto-healing playbooks (Ansible)** triggered by alert rules. **Jenkins pipelines** were enhanced with health-check rollbacks. This reduced downtime and improved SLA compliance.

Roles & Responsibilities:

- Deployed **Prometheus federation** for centralized metrics collection across 10+ clusters.
- Built **Grafana dashboards** tracking application SLIs, SLAs, and error budgets.
- Developed **Ansible playbooks** for auto-healing (restarting pods, scaling nodes, re-deploying apps).
- Integrated Prometheus alerts with **Jenkins pipelines** for automated rollback triggers.
- Configured **ELK Stack (Elasticsearch, Logstash, Kibana)** for log aggregation.
- Implemented **custom HPA (Horizontal Pod Autoscaler)** with business-driven metrics.
- Conducted **chaos engineering experiments** to validate auto-healing workflows.
- Configured **alert routing policies** to Slack/Teams for on-call engineers.
- Built **weekly incident review dashboards** for leadership reporting.

Impact / Achievements:

- Reduced **MTTR by 50%** with auto-healing automation.
- Improved SLA compliance from **98% → 99.9%.**
- Reduced incident detection time by **60%** with real-time dashboards.
- Lowered on-call escalations by **30%**, improving team productivity.
- Enabled monitoring of **200+ microservices** across hybrid cloud.
- Strengthened **capacity planning** with predictive scaling dashboards.
- Boosted **customer trust** with transparent SLA dashboards.
- Reduced downtime impact on revenue by **~20% annually.**

Personal Details

Father's Name: Bharat Trivedi
Date of Birth: 15th January, 2003
Languages: English, Hindi
Marital Status: Unmarried
Gender: Male